BUILDING A GAME WITH SEVERAL STAGES

Maps and `Levels`: stages, spritesheets and 3D levels

Spritesheets, tile flags, map layers, linear level tools and 3D maps: from the first tile to the last stage.

Contents

What a level is in bm **5**

What it does 5

Why it matters 5

Good to know 5

Caveats 5

The sprite sheet and the tiles **7**

Where to start 7

Draw the tiles 7

The flags of a tile 7

Zones and boxes 8

The 2D map in bm SDK **9**

What it does 9

How to open it 9

The keys of the map page 9

Layers 9

Paint a stage 10

Flags in the code 10

The linear level tools (path, apples, crates, lamps) **11**

What it does 11

The tools 11

Good to know 11

Draw a path 11

3D levels **13**

What it does 13

The four layers 13

Good to know 13

The placeholders 13

The 3D preview 13

Build a small 3D level 14

Caveats . . . 14

Building a game with the suite . . . 15

The flow . . . 15

The project and the build . . . 15

The files in the folder of a game . . . 15

Caveats . . . 15

Several stages in a 2D game . . . 16

What it does . . . 16

Three ways to build the stages . . . 16

Good to know . . . 16

Make three stages with layers . . . 16

Regions in one big map . . . 16

Caveats . . . 17

Several stages in a 3D game . . . 18

What it does . . . 18

Good to know . . . 18

The functions of the game . . . 18

Caveats . . . 18

A 3D stage in the code . . . 18

Switching levels in the game . . . 20

What it does . . . 20

Good to know . . . 20

A game with three stages . . . 20

Try it, step by step . . . 21

Caveats . . . 21

Tips and limits . . . 22

Limits . . . 22

Tips . . . 22

When something goes wrong . . . 22

What is not there yet . . . 22

Open points **23**

README

What this is. The guide to building the **levels** of a game with the bm suite: how the sprite sheet and the map fit together, how to draw a 2D map, how to build a 3D level, and how a game goes from one stage to the next.
How to use it. Read the first chapter once: it explains what bm gives you for levels and what it leaves to your code. Then follow the numbered steps of the chapter you need. Anything in a bordered box with a shadow is meant to be copied as it is.
What is missing. Some things are not in bm yet, or only partly. They are marked **[IN DEVELOPMENT]** with one line saying what is missing. Nothing is invented.

What a level is in bm

A bm game is one cartridge: one sprite sheet, one map made of layers, optional 3D models and the code. bm has no separate "level file". A level is whatever your code decides to show from the map and the sheet.

What it does

- The **sprite sheet** is the one picture with all the tiles and sprites. You draw it in bm Pixel.
- The **map** is a grid of cell numbers that point to tiles of the sheet. It has 1 to 8 **layers** of the same size, each with a name.
- The **flags** (0 to 7) belong to each tile of the sheet and tell the game what a tile is: wall, platform, ladder, water, danger.
- The **boxes** are rectangles stored with the named zones of the sheet: where a character can be hit and where its blow hurts.
- For **3D**, four special layers (`floor` , `height` , `walls` , `objects`) become real geometry that the game draws and collides with.

Why it matters

You build the world on the console itself, with bm SDK: draw the tiles, paint the map, press F5 and play it. The same file becomes the final `.bm` with Ctrl+B.

Good to know

- The editors work on a **project** (`.bme`). A game (`.bm`) is read only; saving it makes an editable copy.
- A new map is 256x256 cells with one layer.
- The sections of the cartridge that matter here are 11 SPRITES (zones), 12 LAYERS (map), 13 FLAGS (tile flags) and 14 BOXES (hitboxes).
- Several stages are made by your code: it chooses which part or which layer of the map to show, and what to put in it. Chapters 7 and 9 show how.

Caveats

- **[IN DEVELOPMENT]** A list of levels or stages inside the cartridge, with a key to switch between them in the editor, does not exist. What is missing: the bm SDK has one map and no "level 1, level 2" entries. Today the stages are layers or regions of the same map, or are built by your code.
- **[IN DEVELOPMENT]** `lib.world()` (a loader for map, entities and triggers) and a room grid for the map page are only proposals in the design notes of bm SDK. They are not in the suite.

- **[IN DEVELOPMENT]** Importing maps from Tiled is not available. What is missing: the whole import; the map comes from the SDK or from CSV files.

The sprite sheet and the tiles

Where to start

The sheet is drawn in **bm Pixel**: open it from the **Dev** tab of the menu, or press **F3** in bm SDK to see the sprites. The sheet is a picture divided into 8x8 cells. Cell `n` is at column `n % (width/8)` and row `n // (width/8)`. It can be up to 4096x4096 pixels.

Draw the tiles

1. Open your project in bm Pixel.
2. Press **z** until the sprite size is 8. One sprite is now one tile.
3. Draw a floor tile, a wall tile and a ladder tile, each in its own cell. Keep the ones of the same kind close together.
4. Press **F2** to see the whole sheet and check where each tile is.
5. Press **Ctrl+S** to save.

> **NOTE**
> Draw the tiles at size 8. A 16x16 tile is just four cells, which the map editor can place together (see the next chapters).

The flags of a tile

Every tile has 8 flags, numbered 0 to 7. The game reads the flags and not the tile numbers, so you can change the drawing without touching the code. The convention of bmlib, which every game may change:

Flag	Value	Meaning
0	1	solid
1	2	platform: you can pass through from below
2	4	ladder
3	8	water
4	16	hurts
5 - 7		free

Set the flags in **bm SDK**, not in bm Pixel:

1. Open the project in bm SDK and press **F3**: the sprite page.
2. Choose the cell of your wall tile.
3. Press **0** to turn flag 0 on. The keys **0** to **7** are the flags of the cell.
4. Choose the cell of the ladder and press **2**.
5. Press **F3** again for the map and press **c**: the tiles with flags are shown with a coloured frame.

NOTE

The flags belong to the tile of the sheet, not to a place on the map. Every layer and every stage that uses that tile gets the same flags.

Zones and boxes

A **zone** gives a name to a rectangle of the sheet, with its frames and the speed of the animation. The **boxes** are the rectangles of a zone's frames: `hurt` where it can be hit, `hit` where its blow hurts, `body` the room it takes.

- Zones and boxes are written in a text file (`zones.txt`): a zone is `name x y width height [frames [fps]]` , its boxes are the lines under it. The file is built into the cartridge with `mkbm.py --sprites` . bm Pixel shows the boxes over the sprite (key **j**).
- The game reads them with `zspr()` , `zone()` and `zboxes()` .
- **[IN DEVELOPMENT]** Naming a zone and drawing, moving or resizing a box on the console is not possible. What is missing: an editor for zones and boxes; today they are written in the zones text file.

COPY AND PASTE

```
punch   0  64 32 32 4 12
  hurt  *  8  2 16 30
  hit   3  24 10 10  6
```

The 2D map in bm SDK

What it does

The map page of bm SDK paints the map with the tiles of the sheet, on up to 8 layers. It shows the layer you are on, the cell under the cursor and the tile you hold.

How to open it

Press **F3** to see the sprites, then **F3** again: the map. Press **F3** again to go back to the sprites. Hold **F12** to see the keys of the page.

The keys of the map page

Key	Action
arrows	move the cursor
PgUp / PgDn, Home / End	move a page up / down, left / right
Space	place the tile
Backspace	clear the cell
x	pick the tile under the cursor
, and .	the tile before / after
Tab	choose the tile
f	fill
z	8x8 or 16x16 (a tile of 2 x 2 cells, on even cells)
+ / -	zoom in / out
m	the whole 256x256 map on one screen (overview)
l	the next layer
Shift+L	a new layer
r	rename the layer
o	this layer only, or all of them
c	show the flags of the tiles
u	undo

With a mouse: the left button places the tile, the middle button picks it, the wheel scrolls.

Layers

- A map has at most 8 layers: the SDK says `a map has at most 8 layers` .
- The first layer is drawn at the back. The names are yours: `back` , `front` , `stage2` .
- All the layers have the same size.
- The status line shows the layer and its name.

NOTE

In a layer named `objects` the brush becomes 16x16 and the first placeholder tile is chosen. Placeholders are numbers from 1024 up: the game does not draw them, it reads them and builds things there.

Paint a stage

1. Open a project from a template, for example **Platform 2D** (**Ctrl+N**), or your own.
2. Press **F3** twice to see the map.
3. Press **Shift+L** to add a layer. Press **r** and type the name `stage1`.
4. Press **Tab**, choose a floor tile with the arrows and press **Enter**.
5. Move with the arrows and press **Space** to paint the ground.
6. Choose a wall tile, paint the walls, then press **c** to check the flags: solid tiles show a frame.
7. Press **m** to see the whole map, then **m** again to return.
8. Press **u** if you made a mistake.
9. Press **Ctrl+S** to save, then **F5** to try the game.

TRY IT

Add a second layer called `front`, paint tree tops or arches on it, and press **o** to look at this layer only and then at all of them.

Flags in the code

The game reads the flags with `fget`, `mget` and `mflags`.

COPY AND PASTE

```
-- the cell is solid
if fget(mget(cx, cy), 0) then end

-- the character (8x8) stands on something solid
if mflags(px, py + 8, 8, 1) & 1 ~= 0 then on_ground = true end

-- draw only the ladders of layer 1
map(0, 0, 0, 0, 80, 45, 1, 4)
```

The linear level tools (path, apples, crates, lamps)

What it does

On the map page, a few tools save you from placing every cell by hand. They place a line of tiles or an object at a time. They are made for 3D maps, where they work on the layers `floor` and `objects`. Chapter 5 uses them in a full level.

The tools

Key	Tool	What it does
Shift+P	path	a corridor from a start cell to an end cell, on the layer `floor`, with the tile you hold
Shift+W	path width	cycles the width of the corridor, from 1 to 5
Shift+A	apples row	a line of apple placeholders (1026) on the layer `objects`
Shift+C	crate	a crate placeholder (1028) at the cursor, on the layer `objects`
Shift+O	lamp	a lamp placeholder (1032) at the cursor, on the layer `objects`
Shift+S	start	the start placeholder (1024) of the level
Shift+G	goal	the goal placeholder (1030) of the level

Good to know

- Path and apples take two presses of **Enter**: one on the start cell, one on the end cell. The line is drawn between them. A yellow line shows it while you move.
- The tools look for a layer named `floor` or `objects`. If it is missing, they paint on the layer you are on.
- With the 16x16 brush on (**z**) the line moves in steps of two cells.
- One **u** undoes a whole path, row or object.
- The tools are in the menu of the page (the right mouse button).

Draw a path

1. Press **F3** twice. Choose a floor tile with **Tab**.
2. Press **Shift+W** until the message says `path width: 3`.
3. Press **Shift+P**: the message says `path tool (width 3): press Enter at start cell`.
4. Move to the start cell and press **Enter**.

5. Move to the end cell and press **Enter**. The message says `path drawn (width 3)`.
6. Press **u** to take the whole corridor away.

3D levels

What it does

A 3D level is a map whose layers are read by the runtime and turned into floors, walls and objects. The same builder is used by the editor preview and by your game, so what you see is what the game gets.

The four layers

Press **T** on the map page (or *Template: 3D map* in the menu) and the map gets these layers:

Layer	What it holds
`floor`	the tile of the ground; an empty cell is a hole
`height`	the height of the ground, as a number
`walls`	the height of a wall block over the ground
`objects`	placeholders: start, apples, crates, goal, lamps

The template also turns on the 16x16 brush.

Good to know

- A cell of the 3D map is a block of 2x2 tiles. The builder reads the layers at the **top-left tile** of each block. A 16x16 brush with tile 0 gives a floor of holes.
- A height number is multiplied by the step (0.25 by default) to give the height in world units. A walls number is the same, over the ground.
- The editor draws a model as the object if the cartridge has a model with the name of the object (`APPLE` , `CRATE` , `LAMP` , `GOAL` , `PICO`). Without it, it shows a coloured point; lamps give a light.
- The lamp lights of the game are 4 at most (`lamp3d` slots 1 to 4).

The placeholders

Number	Name in the game
1024	`PLAYER_START`
1026	`APPLE`
1028	`CRATE`
1030	`GOAL`
1032	`LAMP`
1034	`CRATE_BOUNCE`

The tile chooser stops at the last tile of the sheet, so the markers have their own keys: **Shift+S**, **Shift+G**, **Shift+A**, **Shift+C**, **Shift+O**. **[IN DEVELOPMENT]** `CRATE_BOUNCE` (1034) is known to the runtime but has no key in the editor: write it from the code with `mset` .

The 3D preview

Press **v** on the map page to switch between the 2D map and the 3D preview.

- left / right turn the camera; up / down move it near and far.
- Shift+Up and Shift+Down tilt it.
- `,` and `.` change the tile you hold.
- The yellow square on the ground is the cursor; a yellow line shows a tool in use.

The status bar counts the apples, crates and lamps and gives an estimated number of triangles for the floor and the objects. Use it as a budget while you build.

Build a small 3D level

1. Press **Ctrl+N** and choose a 3D template, or start from your own project.
2. Press **F3** twice, then **T**: the four layers appear.
3. Choose the layer `floor` with **I**. Choose a floor tile with **Tab**.
4. Press **Shift+P**, **Enter** at one end, **Enter** at the other: a corridor. Repeat to make a room.
5. Choose the layer `height` with **I** and place a tile: its number is the height of the block, in steps. Choose `walls` and do the same for a wall.
6. Press **v**: you see the level. Orbit with the arrows.
7. Press **Shift+S** where the player starts and **Shift+G** at the goal. The markers go to the layer `objects` by themselves.
8. Press **Shift+A**, **Enter**, **Enter** for a row of apples, **Shift+C** for a crate, **Shift+O** for a lamp.
9. Check the triangle estimate in the status bar. Press **Ctrl+S**, then **F5**.

BEWARE

The builder always reads the layer named `objects` for the placeholders. Do not rename it. The layers `floor`, `height` and `walls` may have other names, which you pass to `map3d_build`.

Caveats

- **[IN DEVELOPMENT]** The 3D maps of bm SDK are not closed yet. What is missing: the check that a 3D map is saved and reloaded. Planned for the beta (rev 1).

Building a game with the suite

The flow

1. **bm Pixel**: draw the sheet.
2. **bm SDK**, sprite page (F3): set the flags of the tiles.
3. **bm SDK**, map page (F3 twice): paint the layers.
4. **bm Studio** and **bm Mesh**: make the 3D props (models) if the level is 3D. The cartridge keeps them in the MESH section.
5. **bm SDK**, code page (F2): write the logic of the stages.
6. **F5**: try the game; you come back to the same page.
7. **Ctrl+B**: build the final `.bm`.

The project and the build

- The project is a `.bme` file: sheet, map with layers, flags, zones, boxes, models and the code. Save it with **Ctrl+S**.
- **Ctrl+B** builds the `.bm` of the project (or the `.b16` for the RGB30, according to the target the SDK shows). The `.b16` is a separate edition with its own build.
- The sheet, the map and the flags go into the cartridge in the sections 11, 12, 13 and 14, so `cart_load` and `cart_save` carry them.
- For a game that lives in the repository, the build takes files: `mkbm.py --sheet sheet.png --map map.csv`, and one more layer for each `--map name=file.csv` (up to 8, in the order given).

COPY AND PASTE

```
python3 scripts/mkbm.py --sheet sheet.png --map map.csv --map front=map_front.csv --flags flags.csv --sp
```

The files in the folder of a game

File	What it is
`sheet.png`	the sprite sheet
`map.csv`	the first layer
`map_<name>.csv`	one more layer, named `<name>`
`flags.csv`	the flags: numbers 0 to 255 for cells 0, 1, 2... or `n=flags` for cell `n`
`sprites.txt`	the zones and their boxes

Caveats

- A kernel older than the layers draws the first layer and ignores the others.
- **[IN DEVELOPMENT]** A game cannot yet be fully developed without a PC: some resource files are PC-only. What is missing: managing the resource files on the device. Planned for the beta (rev 1).

Several stages in a 2D game

What it does

Your game goes through stages. bm gives you the pieces; the code chooses which one is on screen. There are three ways, and you can mix them.

Three ways to build the stages

Way	How	Good for
One layer per stage	`stage1`, `stage2`, `stage3` as layers; draw with `map(..., "stage2")`	a few short stages, all in the editor
One big map, regions	stages side by side in one large map; `map()` and the camera look at one region	stages of different sizes, no switch of data
Built by the code	the code writes the cells with `mset` from a table or a string	random or generated stages

Good to know

- All the layers have the same size. `msize(w, h)` sets the size of all of them at once (up to 4194304 cells).
- A map has 8 layers at most. Scenery that goes in front of the character (`front`) takes one as well.
- Collisions follow the layer you give to `lib.tiles`: `lib.tiles({ layer = "stage2" })`.
- The flags are those of the tiles and are the same for every stage.
- The cells with numbers of 1024 and above are placeholders: the code reads them (hero start, enemies, the exit) and does not draw them.

Make three stages with layers

1. In bm SDK, press **F3** twice. With **Shift+L** and **r** make three layers: `stage1`, `stage2`, `stage3`.
2. Paint each stage on its layer. Press **o** to look at one layer only.
3. Paint a tile of your choice (the exit) in the same cell of every stage, and use it in the code.
4. Press **Ctrl+S**.
5. In the code, draw and collide with the layer of the current stage (chapter 9).

Regions in one big map

1. Press **m** to see the whole map. Choose where each stage starts, in cells.
2. Paint stage 1 from cell (0, 0), stage 2 from cell (64, 0), and so on.
3. In the code, pass the first cell of the stage to `map(mx, my, ...)` and to `lib.camera` with `bounds`.

Placeholders in the 2D map (an `objects` layer, 16x16 brush) can mark the start of the hero and the enemies.

Caveats

- The map page shows all the layers or one. **[IN DEVELOPMENT]** A "stage" view that shows the layers of one stage together does not exist. What is missing: groups of layers in the editor.

Several stages in a 3D game

What it does

`map3d_build` makes a 3D world from the layers. A game with several 3D stages makes a new world at each stage, with the layers of that stage.

Good to know

- `map3d_build` takes the names of the layers: `{ floor = "floor2", height = "height2", walls = "walls2" }`. The layer of the objects is always `objects`.
- The map has at most 8 layers. A second stage can have its own `floor`, `height` and `walls` layers, but it shares the layer `objects`.
- A world is a value of the game: keep the one for the stage you are in, and build the next when you change.
- The editor builds the world with the default names only (`floor`, `height`, `walls`).

The functions of the game

Function	What it gives
`map3d_build(opts)`	the world, from the layers
`map3d_draw(w)`	draws the visible chunks of 16x16 tiles
`map3d_height(w, x, z)`	the height of the ground, or `nil` for a hole or outside
`map3d_blocked(w, x, y, z, r)`	`true` if a sphere touches a wall or a cliff
`map3d_objects(w, kind)`	the list of the objects, with `kind`, `x`, `y`, `z`
`map3d_size(w)`	the size of the world

Caveats

- **[IN DEVELOPMENT]** Several complete 3D stages in one cartridge are not supported by the editor. What is missing: one `objects` layer for each stage (it is always `objects`) and a 3D preview of a layer set other than `floor`, `height` and `walls`. Today: write the cells of the next stage with `mset` before `map3d_build`, or ship one cartridge for each stage.
- **[IN DEVELOPMENT]** There is no collision with the models in the objects. What is missing: the collision volumes of the props; use `map3d_blocked` for the map and your own test for the objects.
- A map with heights is a grid: no slopes and no arches.

A 3D stage in the code

COPY AND PASTE

```
local world

function load_stage()
  world = map3d_build({ floor = "floor", height = "height", walls = "walls" })
  for _, o in ipairs(map3d_objects(world, "PLAYER_START")) do
    px, py, pz = o.x, o.y, o.z
  end
end

function _draw()
  zclear()
  camera3d(px, py + 2, pz - 4, 0, -0.3, 60)
  map3d_draw(world)
end
```

NOTE

This is the order the preview of the editor uses: `zclear`, `camera3d`, then `map3d_draw`. The editor also sets a light with `light3d`. For a full 3D game see the 3D volume of the API.

Switching levels in the game

What it does

A game goes from title to stage 1, to stage 2, and so on, to an ending. bmlib has the small machine that does it: `lib.states`.

Good to know

- A state is a table with `enter`, `update`, `draw` and `exit`. `S:go(name, ...)` changes state and passes values to `enter`.
- `lib.clear()` stops every tween, timer, script and jingle: call it when a new level begins.
- `lib.store(k, v)` keeps a value in the save of the cartridge: the best stage, the lives.
- The game can also keep a slot with `save()` and read it with `saved()`.

A game with three stages

COPY AND PASTE

```
local lib = require "bmlib"
local STAGES = { "stage1", "stage2", "stage3" }
local n = 1

local S
S = lib.states({
  title = {
    update = function() if btnp("ok") then S:go("play", 1) end end,
    draw = function() cls(0); lib.printc("PRESS A", 160, 0xFFFFFF) end },
  play = {
    enter = function(s, i)
      lib.clear()
      n = i
      lib.tiles({ layer = STAGES[n], edge = true })
      -- put the hero where the start tile is
    end,
    update = function(s)
      lib.step(hero)
      if hero_on_exit() then
        lib.store("stage", n)
        if n < #STAGES then S:go("play", n + 1) else S:go("end") end
      end
    end,
    draw = function(s)
      cls(0)
      map(0, 0, 0, 0, 80, 45, STAGES[n])
      spr(1, hero.x, hero.y)
    end },
  ["end"] = {
    draw = function() cls(0); lib.printc("THE END", 160, 0xFFFFFF) end },
}, "title")

function _update() lib.update(); S:update() end
function _draw() S:draw() end
```

Try it, step by step

1. Paint three layers `stage1`, `stage2`, `stage3` in bm SDK (chapter 7), each with ground.
2. Paste the code above in the code page (**F2**). Write `hero`, `hero_on_exit` and the start of the hero for your game.
3. Press **F5**. At the title, press the confirm button: stage 1 begins.
4. Reach the exit: stage 2 loads, with its own layer, and `lib.clear()` has stopped the timers of the first.
5. Finish stage 3 to see the end screen. Quit with Start + Select.

> **BEWARE**
> If a layer name is wrong, the call fails with an error. Check the name with `mlayers()` and keep one list of the names in the code.

Caveats

- For a 2D game, the *Platform 2D* template (**Ctrl+N**) already gives a hero that runs and jumps on a map. Start from it and change it.
- **[IN DEVELOPMENT]** There is no such template for 3D. What is missing: a 3D template with a menu, several stages and an ending. In 3D, write the title screen, the stage list and the ending by hand.
- For the 3D version the machine is the same: in `enter`, fill the layers of the stage with `mset`, call `map3d_build` and keep the world.

Tips and limits

Limits

What	Limit
Layers of a map	1 to 8
Size of the sheet	multiples of 8, up to 4096
New map	256x256 cells, one layer
Map set by `msize`	up to 4194304 cells
Flags of a tile	8 (0 to 7)
Zones in a file	1 to 1024
Boxes in a file	1 to 4096
Lamps in 3D	4 (`lamp3d` slots 1 to 4)
Path width	1 to 5

Tips

- Draw the tiles at size 8 and keep the same kind of tiles together on the sheet.
- Give the layers names that say what they are, and keep the list of names in one place of the code.
- Try the game with **F5** after each stage. You come back to the same page.
- In 3D, watch the triangle estimate in the status bar while you add objects.
- Keep the stage exit and start tiles in the same cell of every layer, to reuse code.

When something goes wrong

- `a map has at most 8 layers` : remove one before you add another.
- A path or a row is not on the layer you expect: check that the layers are called `floor` and `objects` .
- A 3D floor is full of holes: with the 16x16 brush, tile 0 gives a hole. Choose a tile that is not 0.
- A stage does not collide: check the layer you passed to `lib.tiles` and that the tiles have flag 0.

What is not there yet

- **[IN DEVELOPMENT]** A list of stages in the cartridge and in the editor.
- **[IN DEVELOPMENT]** `lib.world()` and the room grid.
- **[IN DEVELOPMENT]** Tiled import.
- **[IN DEVELOPMENT]** An editor for the boxes.
- **[IN DEVELOPMENT]** An `objects` layer for each 3D stage.
- **[IN DEVELOPMENT]** A key for `CRATE_BOUNCE` (1034) in the editor.

Open points

No open points.